

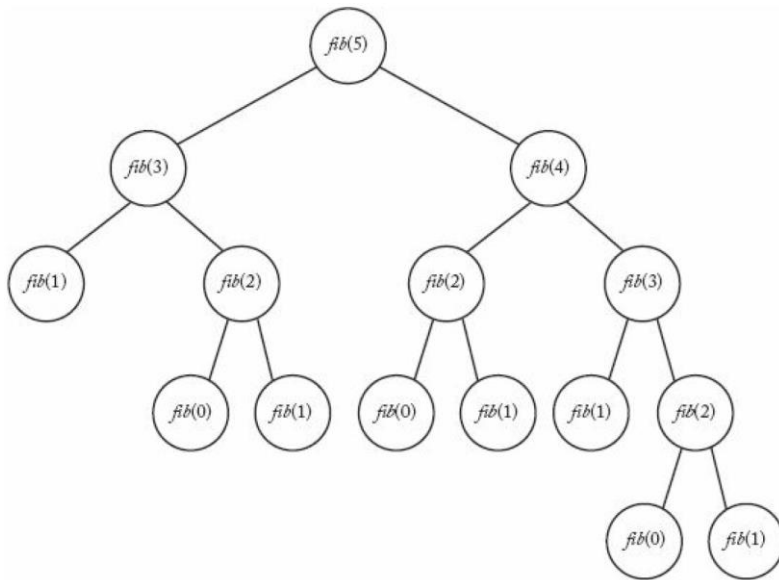
알고리즘

Chap 3. 동적계획 (Dynamic Programming)

■ 동적계획 (Dynamic Programming)

■ 피보나치 수열

- $f_n = f_{n-1} + f_{n-2}$ ($n \geq 2$), $f_0 = 0$, $f_1 = 1$
- 재귀방식 (divide-and-conquer)
 - 항의 개수 $T(n) > 2^{n/2}$ ($n \geq 2$)



- 하향식(top-down) 접근 방법
- 입력 데이터에 관련성이 없는 경우
- 같은 입력값에 대해서 중복 계산

- 반복 방식 (dynamic programming)
 - 항의 개수 $T(n) = n + 1$ ($n \geq 2$)

f(0)	f(1)	f(2)	f(3)	...	f(i)	...	f(n)
------	------	------	------	-----	------	-----	------

- 가장 작은 입력값의 답을 먼저 계산하여 저장해 놓고, 필요할 때 사용한다.
- 상향식(bottom-up) 접근 방법

■ 동적계획 기법의 절차

- 재귀 관계식을 구한다.
- 작은 입력값의 결과를 먼저 계산하는 상향식(bottom-up)으로 전체 입력값의 결과를 계산한다.

■ 이항계수(binomial coefficient) 구하기

- 이항정리(binomial theorem)

$$(x + y)^n = \sum_{k=0}^n \binom{n}{k} x^k y^{n-k}$$

- 이항계수(binomial coefficient)

$$\binom{n}{k} = \frac{n!}{k! (n-k)!} \quad (0 \leq k \leq n)$$

- n개 중에서 k개를 뽑는 경우의 수(조합)
- 팩토리얼(factorial): $n! = n \cdot (n-1) \cdot (n-2) \cdot \dots \cdot 2 \cdot 1$ ($0! = 1$)

- 이항계수의 재귀관계식

$$\binom{n}{k} = \begin{cases} \binom{n-1}{k-1} + \binom{n-1}{k} & , 0 < k < n \\ 1 & , k = 0, n \end{cases}$$

■ 이항계수(binomial coefficient) 구하기

■ 이항계수의 재귀관계식

$$\binom{n}{k} = \begin{cases} \binom{n-1}{k-1} + \binom{n-1}{k} & , 0 < k < n \\ 1 & , k = 0, n \end{cases}$$

■ Algo 3.1 Binomial Coefficient (Divide-and-Conquer)

- 문제: 이항계수를 계산하시오
- 입력: $0 \leq k \leq n$ 을 만족하는 정수 n, k
- 출력: 이항계수 $\binom{n}{k}$

```
int bin (int n, int k)
{
    if (k == 0 || n == k)
        return 1;
    else
        return bin(n-1, k-1) + bin(n-1, k);
}
```

- 같은 항을 중복 계산하므로 비효율적이다. (계산하는 항의 개수: $2\binom{n}{k} - 1$)
- 분할정복에서 입력값의 크기와 분할된 입력값의 크기가 별 차이가 없으면 비효율적일 수 밖에 없다.

이항계수(binomial coefficient) 구하기

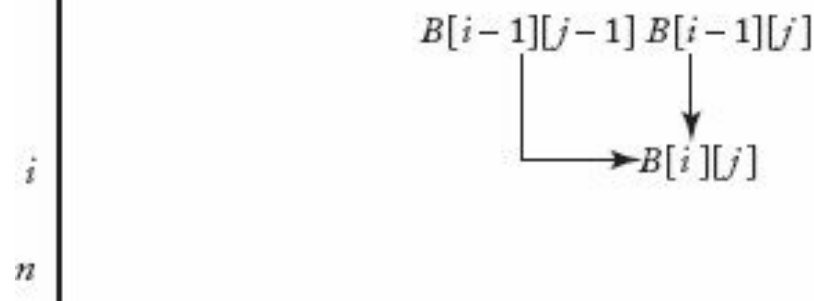
동적계획(dynamic programming) 설계

- 배열 $B[i][j]$ 에 $\binom{i}{j}$ 값을 차례로 저장하여 배열 B 를 만들어 나간다.(bottom-up 방식)
- 각 행에서 k 번째 열까지만 계산한다.

	0	1	2	3	4	j	k
0	1						
1	1	1					
2	1	2	1				
3	1	3	3	1			
4	1	4	6	4	1		

- 재귀 관계식

$$B[i][j] = \begin{cases} B[i-1][j-1] + B[i-1][j], & 0 < j < i \\ 1, & j = 0, i \end{cases}$$



■ 이항계수(binomial coefficient) 구하기

■ Algo 3.2 Binomial Coefficient (Dynamic Programming)

- 문제: 이항계수를 계산하시오
- 입력: $0 \leq k \leq n$ 을 만족하는 정수 n, k
- 출력: 이항계수 $\binom{n}{k}$

```
int bin2 (int n, int k)
{
    index i, j;
    int B[0..n][0..k];

    for (i = 0; i <= n; i++)
        for (j = 0; j <= minimum(i, k); j++)
            if (j == 0 || j == i)
                B[i][j] = 1;
            else
                B[i][j] = B[i-1][j-1] + B[i-1][j];
    return B[n][k];
}
```

■ 이항계수(binomial coefficient) 구하기

■ 효율성 분석

i	0	1	2	3	...	k	$k+1$	...	n
Number of passes	1	2	3	4	...	$k+1$	$k+1$	...	$k+1$

$$\begin{aligned} \bullet \quad T(n, k) &= 1 + 2 + \dots + k + \overbrace{(k+1) + (k+1) + \dots + (k+1)}^{n-k+1} \\ &= \frac{1}{2}k(k+1) + (k+1)(n-k+1) \\ &= \frac{1}{2}(2n-k+2)(k+1) \in \theta(nk) \end{aligned}$$

■ 동적계획 기법

- 입력값을 더 작은 입력값으로 분할하는 재귀관계식을 찾는다
- 작은 입력값부터 시작해서 반복적으로 입력값의 해답을 구한다.
- 작은 입력값의 해답은 한번만 계산하므로 중복 계산을 방지한다.

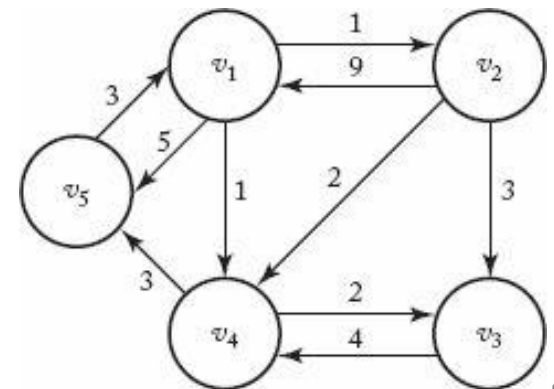
■ 메모리 효율적인 알고리즘

- (algo 3.2)는 인덱스가 0부터 k 까지인 배열 하나만 가지고 계산할 수 있다.

■ 최단경로(shortest path) 알고리즘 (Floyd)

■ 그래프 (graph) $G = (V, E)$

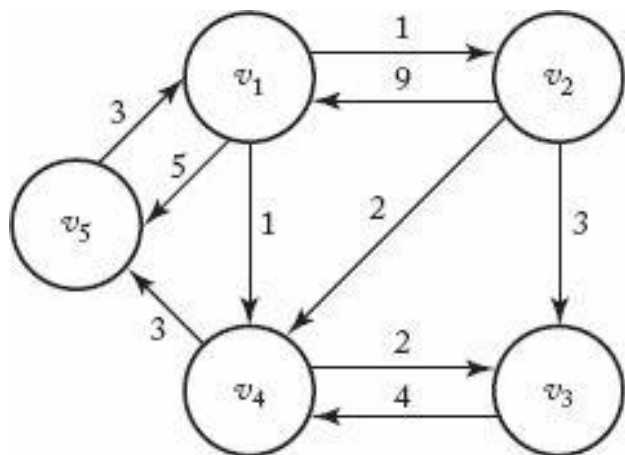
- V: vertex(node)의 집합 $V = \{v_1, v_2, v_3, v_4, v_5\}$
- E: edge(arc)의 집합
- Directed graph(digraph): edge에 방향이 있는 그래프
- Weight: edge의 가중치 (ex. 거리)
- Weighted graph: weight가 있는 그래프
- Path: directed graph에서 edge가 있는 vertex의 배열 (ex. $[v_1, v_4, v_3]$)
- Cycle: 어떤 vertex에서 자기 자신으로 가는 path (ex. $[v_1, v_4, v_5, v_1]$) cf. cyclic vs. acyclic
- Simple path: 같은 vertex를 두 번 거치지 않는 path
 - Simple path는 cycle이 있는 부분 경로(path)를 포함하지 않는다
- Length: weighted graph에서 path에 있는 weigh의 합
unweighted graph에서 path에 있는 edge의 개수
- v_1 에서 v_3 로 가는 simple path
 - $\text{Length}[v_1, v_2, v_3] = 1+3 = 4$
 - $\text{Length}[v_1, v_4, v_3] = 1+2 = 3$ (최단경로)
 - $\text{Length}[v_1, v_2, v_4, v_3] = 1+2+2 = 5$



■ 최단경로(shortest path) 알고리즘 (Floyd)

- 최단경로를 계산하는 가장 쉬운 방법
 - 모든 경로의 길이 중에서 최소값을 찾는 방법
 - 경로의 총 개수: $(n-2) * (n-3) * \dots * 1 = (n-2)! \in \theta(n!)$
- 그래프 표현 방법

$$W[i][j] = \begin{cases} \text{weight on edge} & \text{if there is an edge from } v_i \text{ to } v_j \text{ (adjacent)} \\ \infty & \text{if there is no edge from } v_i \text{ to } v_j \\ 0 & \text{if } i = j. \end{cases}$$



	1	2	3	4	5
1	0	1	∞	1	5
2	9	0	3	2	∞
3	∞	∞	0	4	∞
4	∞	∞	2	0	3
5	3	∞	∞	∞	0

W

adjacent matrix

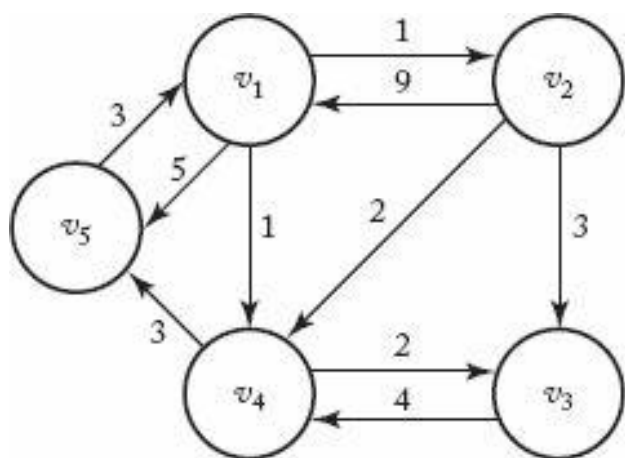
	1	2	3	4	5
1	0	1	3	1	4
2	8	0	3	2	5
3	10	11	0	4	7
4	6	7	2	0	3
5	3	4	6	4	0

D

Vertex 사이의
최단 경로

■ 최단경로(shortest path) 알고리즘 (Floyd)

$D^{(k)}[i][j]$ = 집합 $\{v_1, v_2, \dots, v_k\}$ 에 있는 vertex 만을 거쳐서
 v_i 에서 v_j 로 가는 최단경로의 길이



$$D^{(0)}[2][5] = \text{length}[v_2, v_5] = \infty$$

$$D^{(1)}[2][5] = \text{MIN}(\text{length}[v_2, v_5], \text{length}[v_2, v_1, v_5]) = 14$$

$$D^{(2)}[2][5] = D^{(1)}[2][5] = 14$$

$$D^{(3)}[2][5] = D^{(2)}[2][5] = 14$$

$$D^{(4)}[2][5] = \text{MIN}(\text{length}[v_2, v_1, v_5], \text{length}[v_2, v_4, v_5], \\ \text{length}[v_2, v_1, v_4, v_5], \text{length}[v_2, v_3, v_4, v_5]) = 5$$

$$D^{(5)}[2][5] = D^{(4)}[2][5] = 5$$

$$\begin{array}{c} D^0, D^1, D^2, \dots, D^n. \\ \uparrow \qquad \qquad \qquad \uparrow \\ W \qquad \qquad \qquad D \end{array}$$

1. $D^{(k-1)}$ 에서 $D^{(k)}$ 를 계산하는 재귀관계식을 정한다
2. $k=1$ 부터 n 까지 상향식으로 반복하여 문제를 해결한다

■ 최단경로(shortest path) 알고리즘 (Floyd)

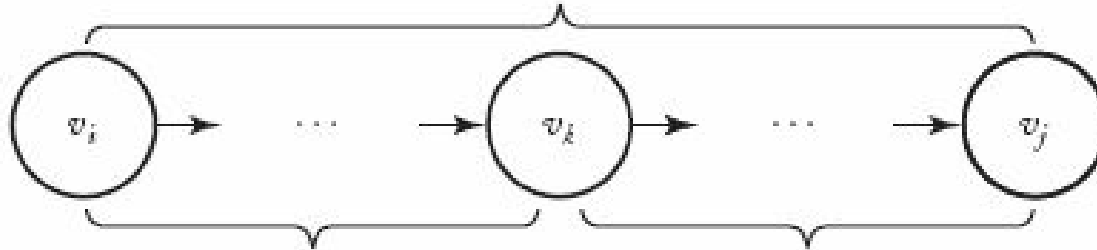
- $D^{(k-1)}$ 에서 $D^{(k)}$ 를 계산하는 재귀관계식을 정한다

- (case 1) $\{v_1, v_2, \dots, v_k\}$ 를 중간 vertex로 거쳐서 v_i 에서 v_j 로 가는 최단 경로들이 모두 v_k 를 거치지 않는 경우

$$D^{(k)}[i][j] = D^{(k-1)}[i][j]$$

- (case 2) $\{v_1, v_2, \dots, v_k\}$ 를 중간 vertex로 거쳐서 v_i 에서 v_j 로 가는 최단 경로들이 모두 v_k 를 거치는 경우

$\{v_1, v_2, \dots, v_k\}$ 에 있는 vertex를 거쳐서 v_i 에서 v_j 로 가는 최단 경로



$\{v_1, v_2, \dots, v_{k-1}\}$ 에 있는 vertex를 거쳐서
 v_i 에서 v_k 로 가는 최단 경로

$\{v_1, v_2, \dots, v_{k-1}\}$ 에 있는 vertex를 거쳐서
 v_k 에서 v_j 로 가는 최단 경로

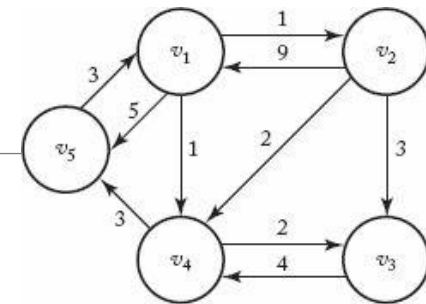
$$D^{(k)}[i][j] = D^{(k-1)}[i][k] + D^{(k-1)}[k][j]$$

- 재귀 관계식

$$D^{(k)}[i][j] = \text{minimum}(D^{(k-1)}[i][j], D^{(k-1)}[i][k] + D^{(k-1)}[k][j])$$

■ 최단경로(shortest path) 알고리즘 (Floyd)

- D^0 부터 상향식으로 D^5 까지 계산



D^0	1	2	3	4	5
1	0	1	∞	1	5
2	9	0	3	2	∞
3	∞	∞	0	4	∞
4	∞	∞	2	0	3
5	3	∞	∞	∞	0

D^1	1	2	3	4	5
1	0	1	∞	1	5
2	9	0	3	2	14
3	∞	∞	0	4	∞
4	∞	∞	2	0	3
5	3	4	∞	4	0

D^2	1	2	3	4	5
1	0	1	4	1	5
2	9	0	3	2	14
3	∞	∞	0	4	∞
4	∞	∞	2	0	3
5	3	4	7	4	0

D^3	1	2	3	4	5
1	0	1	4	1	5
2	9	0	3	2	14
3	∞	∞	0	4	∞
4	∞	∞	2	0	3
5	3	4	7	4	0

D^4	1	2	3	4	5
1	0	1	3	1	4
2	9	0	3	2	5
3	∞	∞	0	4	7
4	∞	∞	2	0	3
5	3	4	6	4	0

D^5	1	2	3	4	5
1	0	1	3	1	4
2	8	0	3	2	5
3	10	11	0	4	7
4	6	7	2	0	3
5	3	4	6	4	0

■ 최단경로(shortest path) 알고리즘 (Floyd)

■ Algo 3.3 Floyd's Algorithm for Shortest Paths

- 문제: 각 vertex에서 다른 모든 vertex로 가는 최단 경로의 길이 계산 (weight≥0)
- 입력: vertex 개수가 n개인 weighted, directed graph (2차원 배열 W)
- 출력: 각 vertex의 최단 경로 길이 (2차원 배열 D)

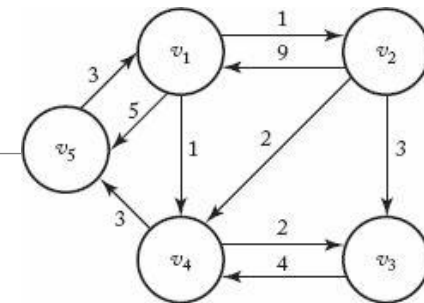
```
void floyd (int n
            const number W[][],
            number D[][])
{
    index i, j, k;
    D = W;
    for (k = 1; k <= n; k++)
        for (i = 1; i <= n; i++)
            for (j = 1; j <= n; j++)
                D[i][j] = minimum(D[i][j], D[i][k] + D[k][j]);
}
```

■ Algo 3.3의 일정 시간 복잡도(Every-case time complexity) 분석

- 단위 연산: for-루프 안의 명령문
- 입력 크기: 그래프의 vertex 개수 n

$$T(n) = n * n * n = n^3 \in \theta(n^3)$$

■ 최단경로(shortest path) 알고리즘 (Floyd)



■ Algo 3.4 Floyd's Algorithm for Shortest Paths 2

- 문제: (Algo 3.3과 동일) 추가로, 최단 경로 찾기
- 추가적인 출력: 최단 경로를 위한 2차원 배열 P는

$$P[i][j] = \begin{cases} v_i \text{에서 } v_j \text{ 가는 최단 경로의 중간에 vertex가 있는 경우, 그 중 가장 큰 index} \\ v_i \text{에서 } v_j \text{ 가는 최단 경로 중간에 vertex가 없는 경우, 0} \end{cases}$$

```
void floyd2 (int n,
             const number W[][],
             number D[][],
             index P[][])
{
    index i, j, k;

    for (i = 1; i <= n; i++)
        for (j = 1; j <= n; j++)
            P[i][j] = 0;

    D = W;
    for (k = 1; k <= n; k++)
        for (i = 1; i <= n; i++)
            for (j = 1; j <= n; j++)
                if (D[i][k] + D[k][j] < D[i][j]){
                    P[i][j] = k;
                    D[i][j] = D[i][k] + D[k][j];
                }
}
```

P	1	2	3	4	5
1	0	0	4	0	4
2	5	0	0	0	4
3	5	5	0	0	4
4	5	5	0	0	0
5	0	1	4	1	0

■ 최단경로(shortest path) 알고리즘 (Floyd)

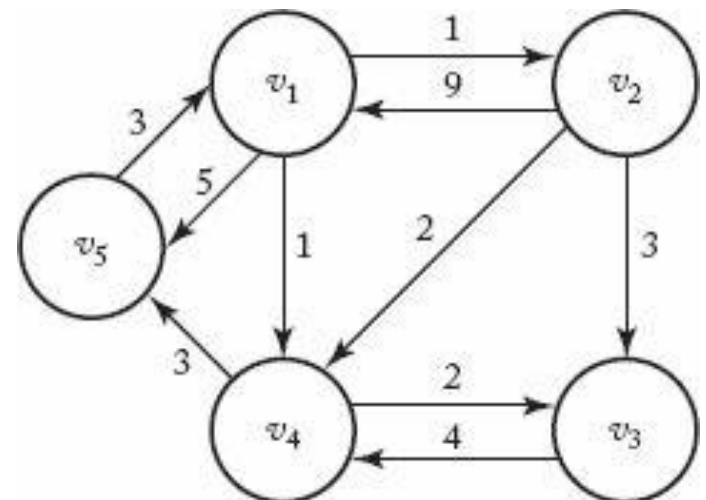
■ Algo 3.5 Print Shortest Path

- 문제: weighted graph에서 최단 경로 중간에 있는 vertex 출력
- 입력: 배열 P (algo 3.4), 두 개 vertex의 인덱스 q, r
- 출력: v_q 에서 v_r 로 가는 최단 경로상의 중간 vertex

```
void path (index q, r)
{
    if ( $P[q][r] \neq 0$ ) {
        path(q,  $P[q][r]$ );
        cout << "v'" <<  $P[q][r]$ ;
        path( $P[q][r]$ , r);
    }
}
```

■ Example

- $\text{path}(5, 3) \rightarrow v_1, v_4$
- v_5 에서 v_3 으로 가는 최단 경로: $[v_5, v_1, v_4, v_3]$



■ 외판원 문제 (traveling salesperson problem)

■ 외판원 문제

- 한 도시에서 출발하여 각 도시를 한 번씩 방문하고, 다시 출발한 도시로 돌아오는 가장 짧은 경로를 찾는 문제
- 일주여행길(tour)
 - 한 도시에서 출발하여 각 도시를 한 번씩 방문하고, 다시 출발한 도시로 돌아오는 경로
- 최적일주여행길(optimal tour)
 - 일주여행길(tour)에서 최소거리 경로

$$\text{length}[v_1, v_2, v_3, v_4, v_1] = 22$$

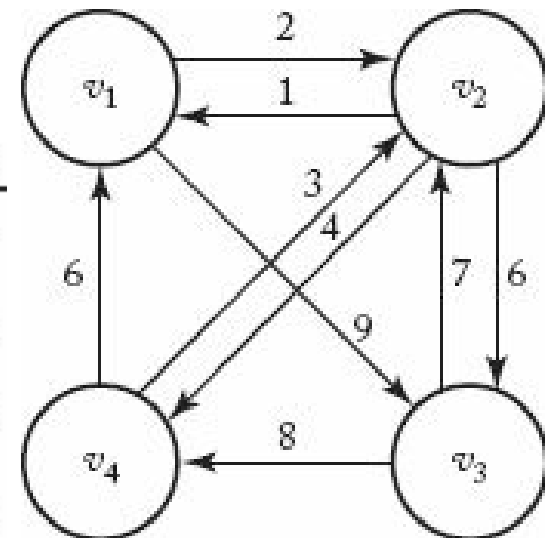
$$\text{length}[v_1, v_3, v_2, v_4, v_1] = 26$$

$$\text{length}[v_1, v_3, v_4, v_2, v_1] = 21 \text{ (optimal tour)}$$

■ 외판원 문제를 계산하는 가장 쉬운 방법

- 모든 일주여행길(tour)을 모두 찾는 방법
 $(n-1) * (n-2) * \dots * 1 = (n-1)! \in \theta(n!)$

W	1	2	3	4
1	0	2	9	∞
2	1	0	6	4
3	∞	7	0	8
4	6	3	∞	0



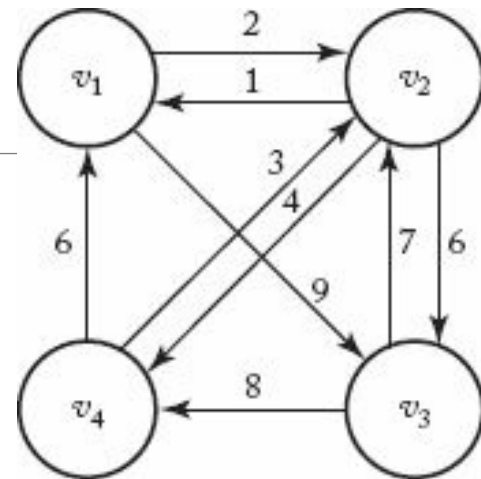
■ 외판원 문제

■ Notation

V = 모든 vertex(도시)의 집합

$A = V$ 의 부분집합

$D[v_i][A]$ = A 에 속한 vertex를 한번씩만 거쳐서
 v_i 에서 v_1 으로 가는 최단경로의 길이



■ Example

$A = \{v_3\}$ 이면, $D[v_2][A] = \text{length}[v_2, v_3, v_1] = \infty$

$A = \{v_3, v_4\}$ 이면, $D[v_2][A] = \min(\text{length}[v_2, v_3, v_4, v_1], \text{length}[v_2, v_3, v_4, v_1]) = 20$

■ 최적일주여행길(optimal tour)의 길이

$$D[v_1][V - \{v_1\}] = \underset{2 \leq j \leq n}{\text{minimum}} (W[1][j] + D[v_j][V - \{v_1, v_j\}])$$

■ 재귀식 ($i \neq 1, v_i \notin A$)

$$D[v_i][A] = \underset{j: v_j \in A}{\text{minimum}} (W[i][j] + D[v_j][A - \{v_j\}]), A \neq \emptyset$$

$$D[v_i][\emptyset] = W[i][1]$$

■ 외판원 문제 (traveling salesperson problem)

■ Example

- $A=\emptyset$: $D[v_2][\emptyset] = 1$, $D[v_3][\emptyset] = \infty$, $D[v_4][\emptyset] = 6$
- $A=\{v_2\}$: $D[v_3][A] = 7+1 = 8$ $D[v_4][A] = 3+1 = 4$
- $A=\{v_3\}$: $D[v_2][A] = 6+\infty = \infty$ $D[v_4][A] = \infty+\infty = \infty$
- $A=\{v_4\}$: $D[v_2][A] = 4+6 = 10$ $D[v_3][A] = 8+6 = 14$
- $A=\{v_2, v_3\}$: $D[v_4][A] = \text{minimum}(3+\infty, \infty+8) = \infty$
- $A=\{v_2, v_4\}$: $D[v_3][A] = \text{minimum}(7+10, 8+4) = 12$
- $A=\{v_3, v_4\}$: $D[v_2][A] = \text{minimum}(6+14, 4+\infty) = 20$
- $A=\{v_2, v_3, v_4\}$: $D[v_1][A] = \text{minimum}(2+20, 9+12, \infty+\infty) = 21$

■ $A=\{v_2, v_3, v_4\}$: $D[v_1][A] = 21$ 세부 계산

$$\begin{aligned} D[v_1][\{v_2, v_3, v_4\}] &= \underset{j:v_j \in \{v_2, v_3, v_4\}}{\text{minimum}} (W[1][j] + D[v_j][\{v_2, v_3, v_4\} - \{v_j\}]) \\ &= \text{minimum} (W[1][2] + D[v_2][\{v_3, v_4\}], \\ &\quad W[1][3] + D[v_3][\{v_2, v_4\}], \\ &\quad W[1][4] + D[v_4][\{v_2, v_3\}]) \\ &= \text{minimum} (2+20, 9+12, \infty+\infty) = 21 \end{aligned}$$

■ 외판원 문제 (traveling salesperson problem)

- Algo 3.11 Dynamic Programming Algorithm for Traveling salesperson problem
 - 문제: Digraph에서 최적일주여행길(optimal tour)을 결정하시오
 - 입력: W (weight를 포함한 digraph), n (vertex의 개수)
 - 출력: minlength (최적일주여행길의 길이), P ($P[i][A]$ 는 A 에 속한 vertex를 한번만 거쳐서 v_i 에서 v_1 으로 가는 최단경로에서 v_i 다음에 오는 첫번째 vertex의 index)

```
void travel (int n, const number W[][], index P[][],
             number& minlength)
{
    index i, j, k; number D[1..n][subset of  $V - \{v_1\}$ ];

    for (i = 2; i <= n; i++)
        D[i][ $\emptyset$ ] = W[i][1];
    for (k = 1; k <= n - 2; k++)
        for (all subsets  $A \subseteq V - \{v_1\}$  containing  $k$  vertices)
            for (i such that  $i \neq 1$  and  $v_i$  is not in  $A$ ) {
                D[i][A] = minimum (W[i][j] + D[j][A - { $v_j$ }]);
                           j:  $v_j \in A$ 
                P[i][A] = value of j that gave the minimum;
            }
    D[1][ $V - \{v_1\}$ ] = minimum (W[1][j] + D[j][ $V - \{v_1, v_j\}$ ]);
                       2 ≤ j ≤ n
    P[1][ $V - \{v_1\}$ ] = value of j that gave the minimum;
    minlength = D[1][ $V - \{v_1\}$ ];
}
```

■ 외판원 문제 (traveling salesperson problem)

- 최적일주여행길(optimal tour) 찾기 예시
 - $P[i][A]$ 는 A 에 속한 vertex를 한번만 거쳐서 v_i 에서 v_1 으로 가는 최단경로에서 v_i 다음에 오는 첫번째 vertex의 index
 - $P[1][\{v_2, v_3, v_4\}] = 3$, $P[3][\{v_2, v_4\}] = 4$, $P[4][\{v_2\}] = 2$
 - 최적일주여행길(optimal tour) : $[v_1, v_3, v_4, v_2, v_1]$
- Algo 3.11의 일정 시간 복잡도(Every-case time complexity) 분석
 - 단위 연산: (for-루프의 minimum 연산에서) 각 v_j 에 대한 연산을 실행하는 명령문
 - 입력 크기: 그래프의 vertex 개수 n
$$T(n) = \sum_{k=1}^{n-2} (n-1-k)k \binom{n-1}{k} = (n-1)(n-2)2^{n-3} \in \theta(n^2 2^n)$$
 - 시간 복잡도가 지수보다 더 좋은 알고리즘을 찾은 사람도 없고, 그러한 알고리즘이 없다고 증명한 사람도 없다.
- Example
 - 20개 도시(vertex)에 대한 최적일주여행길 찾기 (단위연산 처리 시간을 $1\mu s$ 가정)
 - 무작위 알고리즘: $19! \mu s = 3857$ 년
 - 동적계획 알고리즘: $(20-1)(20-2)2^{20-3} \mu s = 45$ 초